

Υπολογιστική Νοημοσύνη

Ιωάννης Γ. Τσούλος

Τμήμα Πληροφορικής και τηλεπικοινωνιών
Πανεπιστήμιο Ιωαννίνων

2023

Περίληψη

- 1 Εισαγωγικά θέματα
- 2 Μάθηση συναρτήσεων
- 3 Κατασκευή Χαρακτηριστικών
- 4 Κατασκευή ΤΝΔ

- 1 Η τεχνική της γραμματικής εξέλιξης (grammatical evolution) είναι μια σχετικά πρόσφατη υλοποίηση γενετικού προγραμματισμού με την χρήση γραμματικών BNF.
- 2 Σε αυτήν την υλοποίηση γίνεται παραγωγή προγραμμάτων σε οποιαδήποτε γλώσσα με την χρήση της BNF γραμματικής που βρίσκεται πίσω από την προς υλοποίηση γλώσσα.
- 3 Η μέθοδος αυτή εφαρμόστηκε σε πολλά προβλήματα στην σύγχρονη βιβλιογραφία.

Παραγωγή προγραμμάτων με Γραμματική Εξέλιξη

- Ο αλγόριθμος παραγωγής εκφράσεων ξεκινά από το σύμβολο εκκίνησης της γραμματικής (Start Symbol) και βαθμιαία παράγει την τελική έκφραση αντικαθιστώντας μη τερματικά σύμβολα με τον αντίστοιχο κανόνα παραγωγής. Η επιλογή του κανόνα παραγωγής γίνεται σε δύο βήματα:
 - 1 Επιλογή του επόμενο στοιχείου V από το χρωμόσωμα
 - 2 Επιλέγεται ο αντίστοιχος κανόνας παραγωγής σύμφωνα με το σχήμα: $\text{Rule} = V \bmod NR$, όπου NR είναι το πλήθος των κανόνων παραγωγής που διαθέτει το μη τερματικό σύμβολο που θα αντικατασταθεί.

Wrapping effect

Η διαδικασία παραγωγής προγραμμάτων θα συνεχιστεί μέχρι να συμβεί ένα από τα ακόλουθα:

- 1 Να έχει παραχθεί ένα έγκυρο πρόγραμμα στην αντικειμενική γλώσσα προγραμματισμού. Έγκυρο θεωρείται εκείνο το πρόγραμμα το οποίο δεν έχει άλλα μη τερματικά σύμβολα.
- 2 Να έχουν τελειώσει τα στοιχεία του χρωμοσώματος και να μην έχει ακόμα παραχθεί έγκυρο πρόγραμμα στην αντικειμενική γλώσσα προγραμματισμού. Σε αυτήν την περίπτωση είτε μπορούμε να θεωρήσουμε πως απέτυχε η διαδικασία και να ανατεθεί μια πολύ μεγάλη καταλληλότητα στο χρωμόσωμα ώστε να μην συμμετάσχει σε διαδικασία διασταύρωσης είτε μπορούμε να εφαρμόσουμε αναδίπλωση (wrapping effect) που σημαίνει πως η διαδικασία της παραγωγής ξεκινά και πάλι από την αρχή του χρωμοσώματος. Στις περισσότερες εφαρμογές 1-2 αναδιπλώσεις είναι αρκετές.

Παράδειγμα γραμματικής

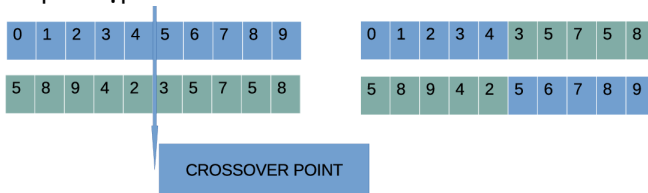
```
<S>::=if(<BEXPR>) CLASS=0 else CLASS=1 (0)
<BEXPR>::=<XLIST><BOOLOP><EXPR> (0)
        |!(<BEXPR>) (1)
        |<XLIST><BOOLOP><EXPR>&<BEXPR> (2)
        |<XLIST><BOOLOP><EXPR>|<BEXPR> (3)
<BOOLOP>::= > (0)
        | >= (1)
        | < (2)
        | <= (3)
<EXPR>::= (<EXPR><BINARYOP><EXPR>) (0)
        | <FUNCTION>(<EXPR>) (1)
        | <TERMINAL> (2)
<BINARYOP>::= + (0)
        | - (1)
        | * (2)
        | / (3)
<FUNCTION>::= sin | cos | exp | log (0-3)
<TERMINAL>::= <XLIST> (0)
        | <DIGITLIST>.<DIGITLIST> (1)
        | (-<DIGITLIST>.<DIGITLIST>) (2)
        <XLIST>::= x1 | x2 | ...|xD (0-D-1)
<DIGITLIST>::=<DIGIT> (0)
        | <DIGIT><DIGIT> (1)
        | <DIGIT><DIGIT><DIGIT> (2)
<DIGIT>::= 0 | 1 | 2 | 3 | 4 |5 |6 |7 |8 |9 (0-9)
```

Παράδειγμα παραγωγής

String	Selected	Operation
if(<BEXPR>) CLASS=0 else CLASS=1		
if(<XLIST><BOOLOP><EXPR>) CLASS=0 else CLASS=1	10	10%4=0
if(x1<BOOLOP><EXPR>) CLASS=0 else CLASS=1	65	65%2=1
if(x1><EXPR>) CLASS=0 else CLASS=1	12	12%4=0
if(x1><EXPR><BINARYOP><EXPR>) CLASS=0 else CLASS=1	31	31%3=1
if(x1><FUNCTION>(<EXPR><BINARYOP><EXPR>) CLASS=0 else CLASS=1	28	28%3=1
if(x1>cos(<EXPR><BINARYOP><EXPR>) CLASS=0 else CLASS=1	9	9%4=1
if(x1>cos(<TERMINAL><BINARYOP><EXPR>) CLASS=0 else CLASS=1	8	8%3=2
if(x1>cos(<XLIST><BINARYOP><EXPR>) CLASS=0 else CLASS=1	6	6%3=0
if(x1>cos(x0)<BINARYOP><EXPR>) CLASS=0 else CLASS=1	10	10%2=0
if(x1>cos(x0)*<EXPR>) CLASS=0 else CLASS=1	6	6%4=2
if(x1>cos(x0)*<TERMINAL>) CLASS=0 else CLASS=1	2	2%3=2
if(x1>cos(x0)*<XLIST>) CLASS=0 else CLASS=1	0	0%3=0
if(x1>cos(x0)*x1) CLASS=0 else CLASS=1	1	1%2=1

Διασταύρωση στην Γραμματική Εξέλιξη

- 1 Συνήθως γίνεται διασταύρωση ενός σημείου
- 2 Παράδειγμα:



Μετάλλαξη στην Γραμματική Εξέλιξη

- Συνήθως γίνεται με τυχαία αλλαγή ακέραιων τιμών
- Μπορεί να γίνει και με πιο εξελεγμένες τεχνικές

- Δίνεται μια σειρά από σημεία και αναζητείται η συνάρτηση που διέρχεται από αυτά
- Γενικά μπορεί να θεωρηθεί πρόβλημα ελαχίστων τετραγώνων

Συναρτήσεις σφάλματος

- ① Mean squared error.

$$E_{\text{MSE}} = \frac{1}{M} \sum_{j=1}^M (y_j - t_j)^2$$

- ② Mean absolute error.

$$E_{\text{ABS}} = \frac{1}{M} \sum_{j=1}^M |y_j - t_j|$$

- ③ Maximum squared error.

$$E_{\text{MAX}} = \max_j (y_j - t_j)^2$$

Γραμματική για μάθηση συναρτήσεων

$\langle S \rangle ::= \langle \text{expr} \rangle$

$\langle \text{expr} \rangle ::= (\langle \text{expr} \rangle)$
 $| \langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle$
 $| \langle \text{func} \rangle (\langle \text{expr} \rangle)$
 $| \langle \text{terminal} \rangle$

$\langle \text{op} \rangle ::= + \mid - \mid * \mid / \mid ^$

$\langle \text{func} \rangle ::= \sin \mid \cos \mid \exp \mid \log \mid \text{abs} \mid \text{sqrt}$

Παραδείγματα

- Για το πρόβλημα των επιδόσεων σε αγώνα μπάσκετ (BK)

$$f(x) = \sin(8.59) \frac{\cos\left(\sqrt{\exp(\sin(x_1))}\right)}{\sqrt{\cos\left(\frac{x_3}{33.1 - 0.245 \cos(x_3 x_2)}\right)}}$$

- Για την συνάρτηση $h(x) = x \sin(x^2)$

$$f(x) = (\sin(x * \text{abs}(1.0) * x) * x)$$

Το πρόβλημα κατασκευής χαρακτηριστικών

- 1 Σε πολλά προβλήματα τα χαρακτηριστικά είναι πολλά σε σχέση με τα διαθέσιμα πρότυπα
- 2 Κατάρα της διάστασης
- 3 Πολλά χαρακτηριστικά είναι προβληματικά λόγω θορύβου
- 4 Πολλά χαρακτηριστικά δεν παίζουν ρόλο στο σύστημα αναγνώρισης προτύπων
- 5 Απαιτείται μείωση χαρακτηριστικών
- 6 Πολλές φορές χρειάζεται να βρούμε και τις κρυμμένες συσχετίσεις στα χαρακτηριστικά

- 1 Αλγόριθμος PCA
- 2 Multi dimensional scaling
- 3 ISOMAP

Χρήση Γραμματικής εξέλιξης

- 1 Δημιουργία γραμματικής BNF, όπως και στην μάθηση συναρτήσεων
- 2 Τίθεται ένας ζητούμενος αριθμός νέων χαρακτηριστικών, έστω K (πχ $K=2$)
- 3 Κάθε χρωμόσωμα είναι κανόνες για την δημιουργία των νέων χαρακτηριστικών
- 4 Το αρχικό σύνολο δεδομένων μεταμορφώνεται στο καινούριο
- 5 Χρησιμοποιείται ένα μοντέλο μάθησης σαν αξιολογητής
- 6 Το σφάλμα του μοντέλου μάθησης είναι η καταλληλότητα του χρωμοσώματος.

Αποτελέσματα (μάθηση κατηγοριών)

DATASET	MLP GEN	FC MLP	GAIN
BALANCE	8.23%	0.30%	96.35%
DERMATOLOGY	10.01%	4.98%	50.25%
GLASS	58.03%	45.84%	21.00%
HAYES ROTH	35.26%	23.26%	34.03%
HEART	25.46%	17.71%	30.44%
IONOSPHERE	13.67%	8.42%	38.41%
PARKINSONS	17.47%	10.10%	36.46%
PIMA	32.98%	23.76%	27.96%
POPFAILURES	7.66%	4.66%	39.16%
SPIRAL	45.71%	26.53%	41.96%
WINE	20.82%	7.31%	64.89%
WDBC	6.32%	3.47%	45.09%
Z_F_S	9.42%	5.52%	41.40%
Z_O_N_F_S	60.38%	31.20%	48.33%
ZO_NF_S	8.06%	4.00%	50.37%

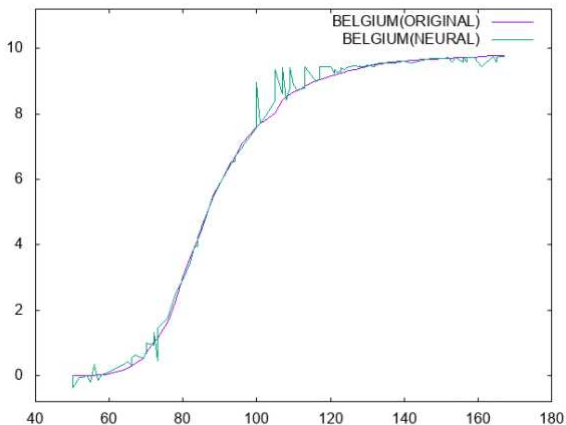
Αποτελέσματα (μάθηση συναρτήσεων)

DATASET	MLP GEN	FC MLP	GAIN
BK	0.21	0.03	85.71%
BL	0.84	0.005	99.40%
Housing	30.05	10.77	64.16%
Laser	0.003	0.002	33.33%
NT	1.11	0.01	99.10%
Quake	0.07	0.03	57.14%
FA	0.04	0.01	75.00%
PY	0.21	0.02	90.47%

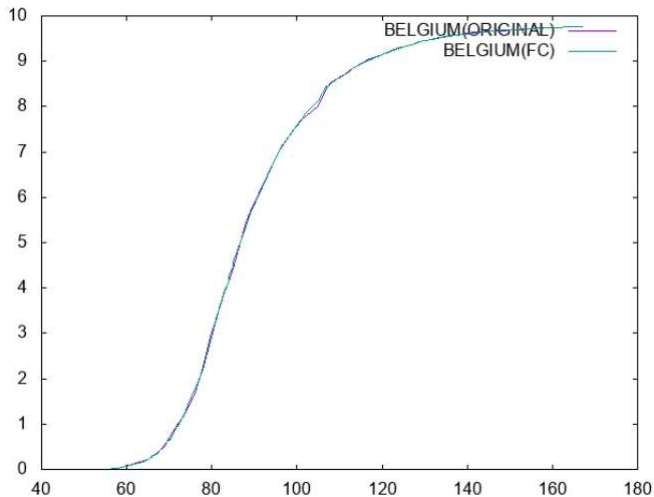
- ➊ Πόσα χαρακτηριστικά θα φτιαχτούν (στην πράξη $K=2$, $K=3$ είναι καλή επιλογή)
- ➋ Ποιος αξιολογητής θα χρησιμοποιηθεί για τα χαρακτηριστικά $\Rightarrow$ χρειάζεται ένα ελαφρύ μοντέλο όπως το RBF
- ➌ Πως τερματίζει ο αλγόριθμος;
- ➍ Υπάρχει και εδώ ο κίνδυνος δημιουργίας μεγάλων αριθμητικών σταθερών

Πρόβλεψη σε COVID

Το απλό ΤΝΔ



Κατασκευή χαρακτηριστικών



Το λογισμικό QFC

- Ελεύθερο λογισμικό δημιουργίας χαρακτηριστικών.
- Διαθέσιμο από <https://github.com/itsoulos/QFc>
- Ο χρήστης μπορεί να διαλέξει μέθοδο εκτίμησης των χαρακτηριστικών
 - Neural
 - RBF
 - KNN
- Ο χρήστης μπορεί να διαλέξει και μοντέλο εκτέλεσης μετά την δημιουργία των χαρακτηριστικών
 - Neural
 - RBF
 - KNN
 - NNC

Figure: Data format.

- | | | | | | |
|---|----------|----------|----------|----------|----------|
| D | | | | | |
| M | | | | | |
| | x_{11} | x_{12} | $\dots$ | x_{1D} | y_1 |
| | x_{21} | x_{22} | $\dots$ | x_{2D} | y_2 |
| | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ |
| | x_{M1} | x_{M2} | $\dots$ | x_{MD} | y_M |

Πειραματικά αποτελέσματα QFC(classification)

DATASET	MRMR	PCA	FCRBF	FCMLP	FCNNC
ALCOHOL	56.75%	70.29%	35.56%	26.57%	28.58%
AUSTRALIAN	32.92%	49.97%	15.37%	14.31%	14.24%
BALANCE	56.80%	56.48%	14.39%	1.42%	1.52%
DERMATOLOGY	68.54%	62.11%	22.42%	15.06%	15.42%
HAYES ROTH	61.21%	61.13%	34.59%	29.58%	30.67%
HEART	38.04%	35.84%	18.61%	15.67%	17.21%
IONOSPHERE	12.93%	21.22%	9.83%	9.48%	9.92%
PARKINSONS	17.16%	16.96%	8.48%	10.82%	12.53%
PIMA	26.29%	39.43%	24.07%	30.02%	25.01%
WDBC	12.91%	10.28%	3.39%	4.36%	4.51%
WINE	30.73%	30.39%	7.61%	11.08%	11.61%
Z_F_S	32.71%	44.81%	5.48%	6.72%	6.63%
ZO_NF_S	33.79%	40.02%	4.08%	4.25%	4.34%

Πειραματικά αποτελέσματα QFC(data fitting)

DATASET	MRMR	PCA	FCRBF	FCMLP	FCNNC
ABALONE	6.48	6.70	4.46	4.18	4.39
AIRFOIL	0.003	0.019	0.002	0.001	0.001
BASEBALL	100.21	101.87	48.04	52.50	51.40
BK	0.03	0.17	0.02	0.02	0.02
BL	0.15	0.19	0.04	0.001	0.01
CONCRETE	0.025	0.273	0.006	0.004	0.005
DEE	0.40	0.55	0.18	0.40	0.38
DIABETES	19.86	27.36	1.49	0.58	0.61
HOUSING	67.97	119.08	12.78	28.47	17.47
FA	0.02	0.08	0.01	0.02	0.01
MB	3.39	4.33	0.48	0.12	0.06
MORTGAGE	0.17	2.41	0.66	1.37	0.22
NT	0.05	1.57	0.25	0.007	0.02
PY	1.56	0.30	0.17	0.03	0.03
QUAKE	1.44	1.80	0.06	0.02	0.04
TREASURY	0.12	3.13	0.29	1.41	0.11

Παράδειγμα εκτέλεσης QFC

```
./QFc --trainFile=examples/wdbc.train  
--testFile=examples/wdbc.test  
--features=2 --ge_maxGenerations=5  
--featureEvaluatedModel=rbf --threads=8
```

Παραγόμενα χαρακτηριστικά (παράδειγμα)

```
• #include <math.h>
void fcMap(double *inx, double *outx)
{
    double x1=inx[0];
    double x2=inx[1];
    outx[0]=(x1*((-2.208)*x1));
    outx[1]=((-6.83)*x1 /
              abs(1.988*x2+1.08*x2+log(x1)));
}
```

Το πρόβλημα

- 1 Πολλές φορές χρησιμοποιούνται εξελικτικές τεχνικές για την μάθηση βαρών στα ΤΝΔ
- 2 Αυτοί οι αλγόριθμοι επιτυγχάνουν πολύ καλά αποτελέσματα
- 3 Το πρόβλημα είναι ότι πολλές φορές δεν έχουν καλές γενικευτικές ικανότητες.
- 4 Χρειάζεται να γίνει μάθηση και της δομής του ΤΝΔ ώστε να είναι αποτελεσματική η μάθηση

- ① Μάθηση βαρών
- ② Εύρεση με γενετικό αλγόριθμο του πλήθους των επιθυμητών κόμβων
- ③ Δημιουργία τοπολογίας με Γενετικό αλγόριθμο
- ④ Γενικά οι περισσότερες υλοποιήσεις αντιμετωπίζουν μόνο μέρος του προβλήματος.

Κατασκευή με Γραμματική Εξέλιξη

- 1 Δημιουργία κατάλληλης γραμματικής
- 2 Κάθε χρωμόσωμα αναπαριστά τους κανόνες για να φτιαχτεί ένα ΤΝΔ
- 3 Το ΤΝΔ δοκιμάζεται στα δεδομένα εκπαίδευσης
- 4 Το μέσο τετραγωνικό σφάλμα είναι η καταλληλότητα του χρωμοσώματος

```

S:=<sigexpr>                                (0)
<sigexpr>::=<Node>                            (0)
           | <Node> + <sigexpr>                (1)
<Node>::=<number>*sig(<sum>+<number>)          (0)
<sum>::= <number>*<xxlist>                    (0)
           |      <sum>+<sum>                 (1)
<xxlist>::= x1                                (0)
           |      x2  (1)
           .....
           |      xn  (n-1)
<number>::= (<digitlist>.<digitlist>)          (0)
           |      (-<digitlist>.<digitlist>) (1)
<digitlist>::= <digit>                       (0)
           | <digit><digitlist>               (1)
<digit>::= 0                                 (0)
           |  1 (1)
           .....
           |  9 (9)
    
```

Παράδειγματα χρήσης

- 1 Μάθηση συναρτήσεων
- 2 Μάθηση δεσμών μεταξύ ατόμων
- 3 Σε Εγκεφαλογραφήματα
- 4 Σε επίλυση διαφορικών εξισώσεων
- 5 Σε δεδομένων από ασθενείς με Πάρκινσον

Πειραματικά αποτελέσματα

PROBLEM	RPROP	BFGS	GENETIC	NNC
Wine	59.93%	48.86%	14.26%	4.44%
Glass	71.37%	53.94%	49.41%	48.01%
Pima	33.19%	36.56%	32.16%	25.00%
Wdbc	32.71%	20.91%	6.87%	4.56%
Circular	44.94%	7.95%	7.84%	6.20%
Spiral	49.06%	45.31%	43.58%	44.90%
Liverdisorder	43.56%	38.86%	35.84%	30.06%
Ionosphere	15.78%	17.08%	16.27%	9.66%

Το πρόγραμμα NNC

- Το πρόγραμμα είναι διαθέσιμο από την σελίδα <https://github.com/itsoulos/NNC>
- Μπορεί να εφαρμοστεί σε δεδομένα ταξινόμησης και σε δεδομένα μάθησης συναρτήσεων.
- Επιτρέπει περιοδική εφαρμογή μεθόδου τοπικής ελαχιστοποίησης
- Μπορεί να χρησιμοποιηθεί και για την επίλυση κάποιων μορφών διαφορικών εξισώσεων.

Παράδειγμα εκτέλεσης NNC

- `./NNC -p BK.train-t BK.train -n 20 -o output.txt`
- Ο χρήστης μπορεί να δώσει και αρχείο όπου θα μπουν τα αποτελέσματα από την μάθηση
- Μπορεί να επιλέξει να γίνονται και ενδιάμεσες εκτελέσεις μεθόδου τοπικής ελαχιστοποίησης για επιμέρους βελτιστοποίηση των βαρών.

Τυπική έξοδος NNC

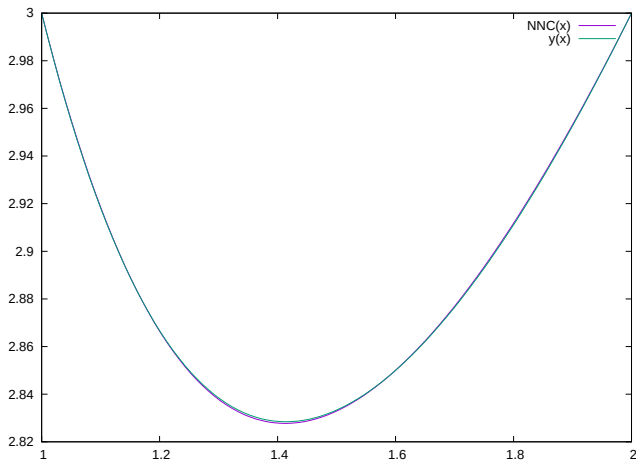
```
1 -1.396837 (0.35)*sig((565.4)*x1+(-6.7))
2 -1.359946 (0.4)*sig((-0.7)*x3+(74.9)*x1+(43.2))
3 -1.331480 (0.4)*sig((-0.76)*x1+(-4.9)*x1+(4.3))
4 -1.319055 (0.4)*sig((-0.76)*x1+(-7.9)*x1+(4.3))
5 -1.319055 (0.4)*sig((-0.76)*x1+(-7.9)*x1+(4.3))
6 -1.299652 (0.5)*sig((-0.76)*x1+(-4.9)*x1+(2.3))
7 -1.299652 (0.5)*sig((-0.76)*x1+(-4.9)*x1+(2.3))
8 -1.295406 (0.5)*sig((-1.76)*x1+(-4.9)*x1+(2.3))
9 -1.293036 (0.5)*sig((-0.76)*x1+(-4.9)*x1+(2.2))
10 -1.293036 (0.5)*sig((-0.76)*x1+(-4.9)*x1+(2.2))
11 -1.292382 (0.5)*sig((-0.74)*x1+(-4.9)*x1+(2.14))
12 -1.292382 (0.5)*sig((-0.74)*x1+(-4.9)*x1+(2.14))
13 -1.292382 (0.5)*sig((-0.74)*x1+(-4.9)*x1+(2.14))
14 -1.292322 (0.5)*sig((-0.74)*x1+(-4.9)*x1+(2.15))
15 -1.292293 (0.5)*sig((-0.78)*x1+(-4.8)*x1+(2.14))
16 -1.292288 (0.5)*sig((-0.66)*x1+(-4.9)*x1+(2.14))
17 -1.292287 (0.5)*sig((-0.74)*x1+(-4.8)*x1+(2.13))
18 -1.292287 (0.5)*sig((-0.74)*x1+(-4.8)*x1+(2.13))
19 -1.292281 (0.5)*sig((-0.73)*x1+(-4.8)*x1+(2.13))
20 -1.292277 (0.5)*sig((-0.71)*x1+(-4.8)*x1+(2.13))
TRAIN ERROR =1.2922774610 TEST ERROR =1.1183353180
SOLUTION: y(x)=(0.5)*sig((-0.71)*x1+(-4.8)*x1+(2.13))
```

Μορφή ODE

```
1  double  getx0(){ return 1; }
2  double  getx1(){ return 2; }
3  int      getkind() { return 1; }
4  int      getnpoints() { return 100; }
5  double  getf0(){ return 3; }
6  double  getff0(){ return 10.0; }
7  double  getf1(){ return sin(10.0);}
8  double  ode1ff(double x,double y,double yy){
9          return yy -(2*x-y)/x;
10 }
11 double  dode1ff(double x,double y,double yy,
12               double dy,double dyy){
13         return dyy+dy/x;
14 }
15 double  ode2ff(double x,double y,double yy,double yyy){
16 }
17 double  dode2ff(double x,double y,double yy,
18               double yyy,double dy,double dyy,double dyyy){
19 }
```

Παράδειγμα επίλυσης διαφορικής εξίσωσης

- $y'(x) = \frac{2x-y}{x}$ with $y(1) = 3$ and $x \in [1, 2]$



Επίλυση διαφορικής εξίσωσης (ode case)

- ODE FORMAT: $\psi(x, y, y^{(1)}, \dots, y^{(n)}) = 0, x \in [a, b]$
- Initial Conditions: $h_i(x, y, y^{(1)}, \dots, y^{(n)})|_{x=t_i}, i = 1, \dots, n$
- ① **Create** $T = \{x_1 = a, x_2, x_3, \dots, x_N = b\}$ a set of equidistant points.
- ② **Create** the NETWORK $r = r(g)$ network for the the chromosome g .
- ③ **Calculate** the value
$$E_r = \sum_{i=1}^N \psi(x_i, r(x_i), r^{(1)}(x_i), \dots, r^{(n)}(x_i))^2$$
- ④ **Calculate** the penalty value for the initial conditions:

$$P_r = \lambda \sum_{k=1}^n h_k^2(x, r(x), r^{(1)}(x), \dots, r^{(n)}(x))|_{x=t_k} \quad (1)$$

where $\lambda > 0$.

- ⑤ **Return** $f(g) = E_r + P_r$

Σύνοψη

- Παρουσιάστηκαν ειδικά θέματα Γραμματικής Εξέλιξης
- Παρουσιάστηκαν μερικοί εξελικτικοί αλγόριθμοι.

Βιβλιογραφία I

-  ITsoulos, I.G., Lagaris, I.E. Solving differential equations with genetic programming. Genet Program Evolvable Mach 7, 33–54 (2006). <https://doi.org/10.1007/s10710-006-7009-y>
-  I. G. Tsoulos, D. Gavrilis and E. Dermatas, GDF: A tool for function estimation through grammatical evolution, Computer Physics Communications 174, pp. 555-559, 2006.
-  I.G. Tsoulos, D. Gavrilis, E. Glavas, “Neural Network Construction and Training using Grammatical Evolution”, Neurocomputing 72, pp. 269-277, 2008.
-  D. Gavrilis, I.G. Tsoulos, E. Dermatas, “Features Selection and Construction using Grammatical Evolution”, Pattern Recognition Letters 29, pp. 1358-1365, 2008.

Βιβλιογραφία II

-  I.G. Tsoulos, D. Gavrilis, E. Glavas, Solving differential equations with constructed neural networks, *Neuromputing* 72, pp. 2385-2391, 2009.
-  Ioannis G. Tsoulos, Creating classification rules using Grammatical Evolution, accepted for publication in *International Journal of Computational Intelligence Studies*, 2018
-  Ioannis G. Tsoulos, Alexandros Tzallas, Dimitrios Tsalikakis, NNC: A tool based on Grammatical Evolution for data classification and differential equation solving, *SoftwareX* 10, 2019.